MAD Backend Implementation Plan

Oct 11, 2026 · @Surya

Summary

We build MAD as two services on Ascension's Google Cloud: a **knowledge graph service** (Phase 1) that every agent queries, and a **delivery runtime** (Phase 2) where a deterministic workflow runs nine steps and calls AI agents as workers. Agents never decide what runs next. The workflow does, and named people approve at H1 to H4 and CAB.

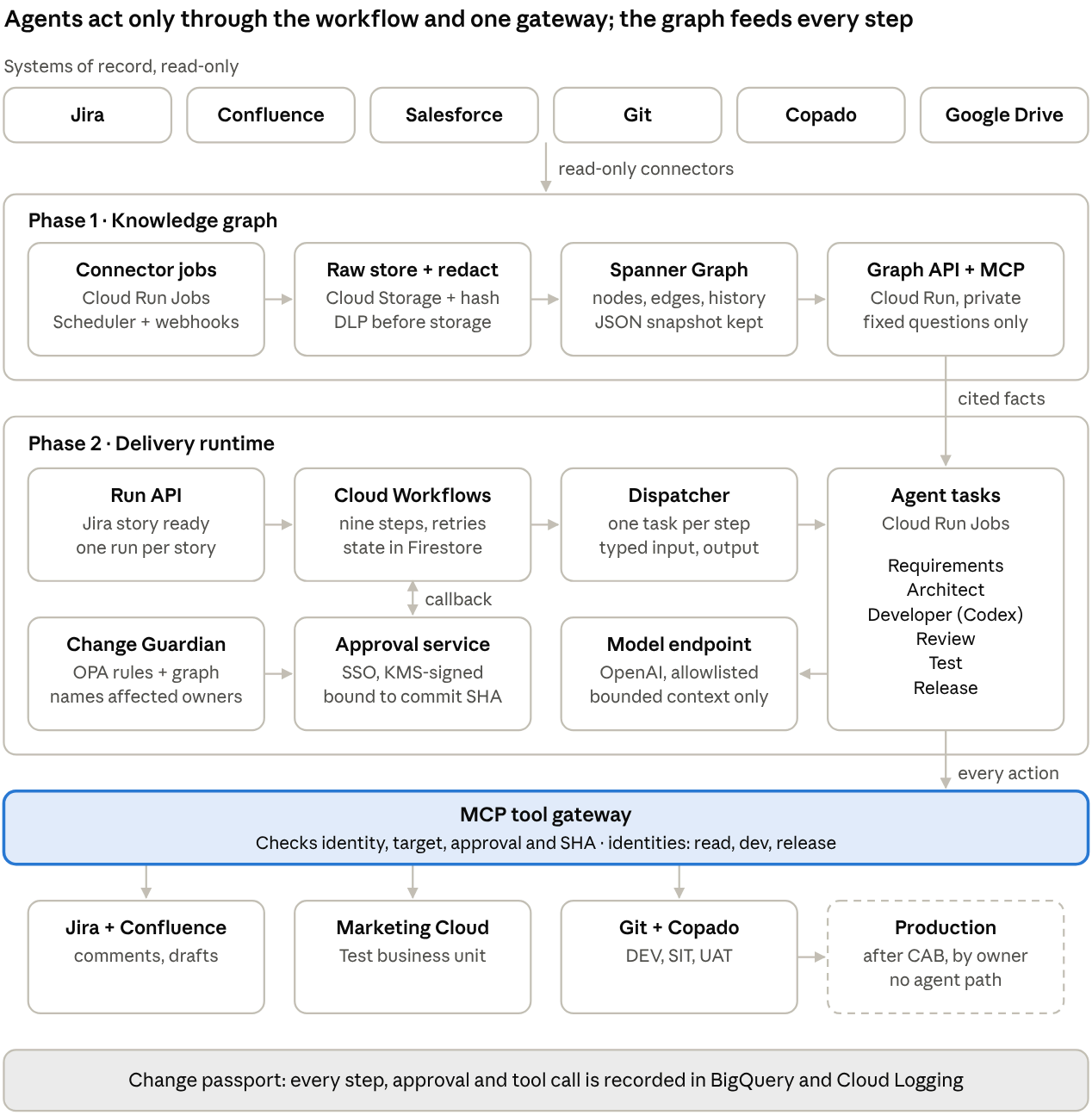
This is a presenter's reference, not a slide. It names each technology, why it was chosen over the alternatives, and what is proven versus still to build. Items marked **Recommendation** are our proposal and need Ascension's architecture and security sign-off in Phase 0.

Decision	Recommendation	Why, in one line
Where it runs	Google Cloud, inside Ascension's existing landing zone	APDH already runs there; one IAM model, one security review
Knowledge graph store	**Spanner Graph** (Enterprise edition), fed from a vendor-neutral JSON snapshot in Cloud Storage	Managed, ACID, ISO GQL, vector and full-text search in one database, inside the GCP perimeter
Graph query layer	Our own **Graph API** (Python, FastAPI on Cloud Run) exposed to agents as an **MCP server**	Agents get fixed, typed questions (impact, reuse, risk), never raw database access
Semantica ContextGraph	Keep its API shape as the contract; do not make it the system of record	In-memory library: fine for the POC, not for concurrent writers, durability or audit (section 3)
Workflow controller	**Google Cloud Workflows** + **Firestore** for run state	Managed, durable, has callbacks that pause for a human approval

Decision	Recommendation	Why, in one line
Agent compute	**Cloud Run Jobs**, one isolated container per agent task	Ephemeral, per-task identity, scales to zero
Coding agent	**OpenAI Codex** (Codex SDK / `codex exec`) in our container	The coding engine the deck names; runs on a branch, writes structured output
Reasoning agents	**OpenAI Agents SDK** (Python) calling the approved OpenAI endpoint	Same vendor, tool calls through MCP, structured outputs
Tool access	**MCP tool gateway** with **Open Policy Agent** rules	Every action checked against identity, target, approval and commit SHA
Audit	**BigQuery** (change passport), **Cloud Storage** (artifacts), **Cloud Logging**	Queryable evidence per change; CAB pack generated, not collated
Infrastructure	**Terraform**, **Cloud Build**, **Artifact Registry**	Repeatable, reviewable, scanned images

What exists today. A Salesforce-sandbox POC of the graph (Apex, JSON stored as a Salesforce File) built 2,177 nodes and 2,786 edges for the Network Services pod and refuses to duplicate a field that already exists. Six of the seven MAD agents are consolidated from agents already used for UI, backend and QA delivery. Everything on GCP described here is the build.

Architecture at a glance



MAD on Google Cloud · Phase 1 graph, Phase 2 runtime, one gateway

Read it top down: sources feed the graph, the workflow launches one agent task per step, agents read cited facts from the graph and act only through the gateway, and production stays with the owning team.

Key technology decisions

The graph store is the one decision that is hard to reverse, so it is compared in full. The rest follow from it and from the rule that everything runs inside Ascension's GCP perimeter.

Knowledge graph store

Recommendation: **Spanner Graph as the query store, a vendor-neutral JSON snapshot in Cloud Storage as the record of each sync.** The JSON contract is the one already designed (deck layers 4 to 6), so the store can change later without rewriting connectors.

Option	What it is	Strengths	Weaknesses for MAD	Verdict
Spanner Graph	Google's managed database with a property-graph layer; queried in ISO GQL; Enterprise edition	Managed in GCP; ACID with many concurrent writers; IAM, CMEK, VPC Service Controls; vector and full-text search in the same database; GQL is an ISO standard	Enterprise edition cost floor; team must learn GQL; graph modelling done in DDL	**Recommended**
Semantica ContextGraph	Python library in the design docs: in-memory directed graph loaded from JSON, with impact, reuse and provenance helpers	Ready-made API shape (load_from_json, get_neighbors, find_path, provenance); fast in memory	Lives in one process: each instance loads the whole graph; no concurrent writes; durability is a JSON file; maturity, support and security posture not yet assessed	Keep its **API as our contract**; not the store

Option	What it is	Strengths	Weaknesses for MAD	Verdict
Neo4j AuraDB	Mature commercial graph database, Cypher, Graph Data Science library	Best graph tooling and algorithms; large community	Third-party vendor: new security review, BAA and data-processing terms; data leaves Google-managed services	Fallback if graph analytics outgrow Spanner
JSON + in-memory (NetworkX) on Cloud Run	What the Salesforce POC does today, moved to GCP	Simplest; free; proven at POC scale	Same limits as Semantica; no history queries; every instance reloads	Use only for the first 2 weeks of Phase 1 to unblock agents
PostgreSQL (AlloyDB) with recursive SQL	Relational tables, traversals as recursive queries	Familiar SQL; pgvector for embeddings	Multi-hop traversals are verbose and slow to write; not a graph model	Not chosen

Why not Semantica as the store, in one sentence for the room: *it is a good library for asking questions of a graph in memory, but MAD needs a durable, shared, audited store that sync jobs, agents and the Change Guardian read and write at the same time.* If Ascension has already licensed Semantica, it can sit on top of Spanner as the analysis layer behind the same Graph API.

Workflow controller

Recommendation: **Google Cloud Workflows** for the run, **Firestore** for run state. Workflows is managed, retries with backoff, and its callback endpoints pause a run until a person approves (the wait has a configurable timeout; default 12 hours). Long approvals re-arm the wait from Firestore state.

Alternative: **Temporal** (self-hosted on GKE or Temporal Cloud). Richer code-first durable execution, but one more platform to run and secure. Move to it only if run logic outgrows Workflows' YAML.

Model and agent runtime

- **Coding:** OpenAI **Codex**, run headless in our own container through the Codex SDK (TypeScript) or `codex exec`, on a dedicated branch, with schema-constrained JSON output.
- **Reasoning agents** (Requirements, Architect, Review, Release): **OpenAI Agents SDK** (Python) against the same approved endpoint, tools reached through MCP.
- **Model access:** OpenAI API under Ascension's agreement (BAA and zero data retention to be confirmed by security), through an allowlisted egress proxy. Because tools sit behind MCP and prompts are versioned, a switch to a model on Vertex AI is configuration plus re-running the evaluation set, not a rebuild.

Language and tooling

Python 3.12 for connectors, Graph API, gateway, Guardian and reasoning agents. TypeScript only where the Codex SDK needs it. Salesforce CLI (`sf`) for every Salesforce action, never raw scripts against production.

Phase 1: building the knowledge graph

Phase 1 delivers one thing agents can trust: a versioned graph of Ascension's delivery estate, refreshed on a schedule and on events, queried through a fixed API. It is built in five stages, each a separate, restartable job.

1. Connectors: who runs them and when

Each connector is a **Cloud Run Job** (Python container). **Cloud Scheduler** starts it on a cron; **Eventarc / Pub/Sub** starts it early when a source signals a change. Every connector reads only what changed since its last high-water mark, stored in Firestore.

Source	How it authenticates (read-only)	What it pulls	Starting cadence	Event trigger
Jira	Atlassian API token or OAuth 2.0, read scopes	Stories, acceptance criteria, links, sprint, status (JQL `updated >= lastSync`)	Every 15 min	Jira webhook → Pub/Sub
Confluence	Atlassian API token, space-scoped	Design pages, business-rule tables, runbooks (storage format)	Hourly	Page-updated webhook
Salesforce	External Client App, JWT bearer, read-only integration user, no record data	Metadata API retrieve, Tooling API (`Flow.Metadata`, `MetadataComponentDependency`), describe()	Nightly full + after each deployment	Copado deployment completed
Git	Read-only deploy key or fine-grained token	force-app tree, Apex, LWC, tests, commits, tags	On push	Git webhook
Copado	Read-only API user	User stories, promotions, deployments, test results	Every 15 min	Pipeline completion
Google Drive	Service account, shared-drive scope	Specs, mapping sheets, evidence (Docs and Sheets exported to text)	Every 30 min	Drive changes API token

Cadences are starting points, tuned in the pilot. A Salesforce read is one record per call for flow bodies (measured in the sandbox: 495 active flows need chained batches), so the Salesforce job pages through flows across several executions.

2. Land raw artifacts

Each item becomes a `SourceArtifact`: bytes + metadata + SHA-256, written to a **Cloud Storage** bucket (`raw/<source>/<id>/<sha256>`), object versioning on, CMEK encrypted. An unchanged hash is skipped, so a nightly run only parses what moved.

3. Redact, parse, validate

A second Cloud Run Job per source:

- **Redact first.** Free text from Jira, Confluence and Drive passes through **Sensitive Data Protection (Cloud DLP)** de-identification before anything is stored parsed. Salesforce is read as metadata only, never records.

- **Parse** into the domain model (deck layer 4): Jira ADF to requirements and Given/When/Then; Confluence HTML to sections, tables and rule sentences; Flow XML to elements, record creates and updates; Apex to SOQL, DML and objects touched; tests to the classes they cover.
- **Validate** with JSON Schema (`pydantic` models): a record that fails is quarantined with a reason, never guessed.
- **Link** with deterministic rules first (a Jira key in a Confluence page, an API name in a story). A link the rules cannot prove is stored as `origin: INFERRED` with a confidence below 1.0 and is never used to clear a change.

4. Write the snapshot and load the graph

- The exporter writes the whole domain context as one **JSON snapshot** (`snapshotId`, nodes, relationships, provenance) to Cloud Storage. This is the portable record and the input to any engine.
- A loader upserts it into **Spanner Graph**: a `Node` table and an `Edge` table (id, label, properties as JSON, source path, SHA-256, `snapshot_id`, `valid_from`, `valid_to`), declared as a property graph. Removed facts get `valid_to` set rather than being deleted, so the graph can answer what it held on the day a change was approved.
- Ownership is a first-class edge: `COMPONENT_OWNED_BY_POD`, declared per pod (as the POC's `AINetworkServicesPod` does) and confirmed by pod leads once.

5. Serve it: Graph API and MCP server

A **Cloud Run service** (FastAPI), private ingress only, called with Google-signed identity tokens. It runs GQL on Spanner and exposes a small, fixed set of questions; agents never write GQL.

Tool	Answers
`graph.exists(type, name, object)`	Does this field, flow, class or rule already exist? Full lookup, never a sample
`graph.find_reuse(capability)`	What already implements this capability?
`graph.blast_radius(node, hops)`	What runs on, calls or depends on this component, behaviour first
`graph.classify_risk(components)`	Risk tier and the approver it routes to

Tool	Answers
`graph.owners(components)`	Which pods own or use each component
`graph.coverage_gap(components)`	Which components have no covering test
`graph.provenance(node)`	Source path, SHA-256, snapshot and retrieval time
`graph.context_slice(story)`	The bounded, cited bundle an agent receives for one story

The same service speaks **MCP** (streamable HTTP), so Codex and the Agents SDK call these as tools. The POC's Salesforce console keeps working by calling this API through a Named Credential.

Phase 1 exit criteria

- All six sources syncing, with freshness and coverage on a dashboard.
- On 10 to 15 representative past changes, `exists`, `blast_radius` and `owners` match what the pod leads say is true.
- Every answer cites node ids and source paths; no PHI found in a DLP scan of the stored graph.

Phase 2: the delivery runtime

Agents do not pick up work and do not talk to each other. A deterministic workflow owns the order of the nine steps; at each step it launches exactly one agent task with a typed input, waits for a typed output, checks it, and only then moves on. This is the single most important design point to say out loud.

How a run starts

1. A story reaches **Ready for MAD** in Jira (a status or label the pod lead sets).
2. The Jira webhook posts to the **Run API** (Cloud Run). It validates the story, creates a run document in Firestore (`runs/{runId}`), and starts one **Cloud Workflows** execution with `runId` as its argument. One story, one run; a second webhook for the same story is ignored (idempotency key = story key + revision).

How an agent knows the next job is its own

It does not need to. The workflow calls the **Dispatcher** with an explicit task:

```
{
  "runId": "run_8f2c", "step": "03-build", "attempt": 1,
  "agent": "developer", "skills": ["health-cloud"],
  "identity": "sa-ai-dev",
  "inputs": ["gs://mad-runs/run_8f2c/02/design.json", "gs://mad-
runs/run_8f2c/02/changeplan.json"],
  "allowedTools": ["graph.*", "git.branch", "git.commit", "sf.deploy_dev",
"sf.test_dev"],
  "budget": {"minutes": 45, "modelTokens": 400000},
  "outputSchema": "build-result.v1"
}
```

The Dispatcher starts a **Cloud Run Job** execution from that agent's container image, passing the task as an environment variable, under the named service account. The container reads its inputs from Cloud Storage, does the work, writes its output JSON to `gs://mad-runs/<runId>/<step>/`, and publishes `step.completed` (or `step.failed`) to **Pub/Sub**. The workflow, waiting on that step, validates the output against its JSON Schema and records a checkpoint in Firestore.

So the routing is plain code in the workflow definition, versioned in Git and reviewed like any other code. The only judgement an LLM makes about routing is in step 01, where the Orchestrator classifies the story (platform, pod, risk), and that classification is itself an output a person approves at H1.

The workflow, in outline

```
main:
  params: [args]
  steps:
    - s01_understand: {call: run_agent, args: {step: "01", agent:
requirements}}
    - h1: {call: await_approval, args: {gate: H1, approver: product_owner}}
    - s02_design: {call: run_agent, args: {step: "02", agent: architect}}
    - guardian_design: {call: guardian_check, args: {point: design}}
    - h2: {call: await_approval, args: {gate: H2, approver:
architect_and_affected_owners}}
    - s03_build: {call: run_agent, args: {step: "03", agent: developer}}
    - s04_verify: {parallel: [review, test], then: fix_loop_max_2}
    - h3: {call: await_approval, args: {gate: H3, approver:
tech_lead_and_security}}
    - s05_promote: {call: run_agent, args: {step: "05", agent: release,
target: UAT}}
    - h4: {call: await_approval, args: {gate: H4, approver: business_owner}}
    - s07_prepare: {call: run_agent, args: {step: "07", agent: release}}
    - cab: {call: await_approval, args: {gate: CAB, approver: release_owner}}
    - s09_close: {call: run_agent, args: {step: "09", agent: orchestrator}}
```

`run_agent` = dispatch, wait for the Pub/Sub completion, validate, checkpoint, with `try / retry` and backoff. `await_approval` = create a Workflows callback endpoint, store it on the run, notify approvers, wait. Step 08's production deployment is not in the workflow at all: the owning team's pipeline runs it after CAB.

Run state (Firestore)

`runs/{runId}`: story key, pod, platform, risk tier, current step, status, attempts per step, token and minute spend against budget, artifact URIs, approval ids, commit SHA and target environment. A run can be inspected or resumed from this document alone.

Artifacts (Cloud Storage)

Every handoff is a file: `brief.json`, `design.json` + `design.md`, `changeplan.json`, `build-result.json`, `findings.json`, `test-report.json`, `release-bundle.json`, `cab-pack.pdf`. Each is versioned, schema-checked and hashed into the change passport. That is what makes every handoff reviewable.

The nine execution steps, implemented

Each row is one workflow step: what runs, what it can touch, what it must hand over, who signs, and what happens when it goes wrong. Nothing advances without the output validating against its schema.

Step	Runs as	Tools and APIs it may call	Must hand over	Gate	When it fails
01 Understand the request	Requirements Agent (Agents SDK) as `sa-ai-read`	Jira read, `graph.context_slice`, `graph.find_reuse`, Confluence search via graph	`brief.json`: scope, acceptance criteria (Given/When/Then), open questions, cited sources; comment posted on the Jira story	**H1** product owner / business lead	Missing source context → open questions go to the story owner; run waits in `NEEDS_INPUT`
02 Design with evidence	Architect Agent (Agents SDK) as `sa-ai-read`, then Change Guardian	`graph.exists`, `graph.blast_radius`, `graph.owners`, `graph.classify_risk`, `graph.coverage_gap`; Confluence page create (draft)	`design.json` + Confluence design page: reuse verdict, Flow vs Apex vs LWC choice with reasons, components to create or modify, risk tier, test scope, affected pods	**H2** architect + every affected pod owner the Guardian names	Graph cannot prove existence or ownership → verdict is `NEEDS REVIEW`, never `safe to create`
03 Build in an isolated workspace	Developer Agent (Codex) as `sa-ai-dev`; Test Agent drafts cases in parallel	Git branch `mad/<story>`, `sf project deploy start` to the DEV sandbox only, `sf apex run test`, Code Analyzer, graph tools	Pull request + `build-result.json` (commit SHA, files changed, local test results)	Guardian on every commit (required CI check)	Compile or test failure → the agent fixes within its budget; budget spent → run pauses for the tech lead
04 Verify independently	Review Agent + Test Agent, in parallel, as `sa-ai-read` + `sa-ai-dev`	PR diff, Salesforce Code Analyzer (PMD rules), ESLint and Jest for LWC, Apex tests in DEV, Zephyr Scale API for cases	`findings.json` (each finding with file, line, rule, severity) + `test-report.json`, both bound to the SHA	**H3** tech lead + security / domain approvers	Blocking findings → fix loop to step 03, at most 2 rounds, then the tech lead decides
05 Promote and prove in UAT	Release Agent as `sa-ai-release`	Copado API: user story, commit, promotion to SIT then UAT; Copado Robotic Testing and regression runs	UAT build id, test evidence, defect list	Guardian at promotion: no passport, no promotion	Promotion or test failure → back to 03 with the evidence; environment issue → release owner
06 Accept the business outcome	People, not an agent; the run waits	Approval page, Jira	UAT sign-off recorded against the SHA	**H4** business UAT + domain owners	Defects are logged on the same run and sent back to 03

Step	Runs as	Tools and APIs it may call	Must hand over	Gate	When it fails
07 Prepare a release that can recover	Release Agent as `sa-ai-release`	Git tags, Copado, graph drift check	`release-bundle.json` (manifest, environment mappings), runbook, rollback plan, CAB pack	None; cannot reach CAB without a rollback plan	Source drift since UAT → back to 05
08 Authorize production release	People: CAB and the platform release owner	Owner's own CI/CD and Copado production pipeline	CAB decision, deployment record	**CAB**	The release owner runs the rollback plan; no agent holds production credentials
09 Observe, learn and close	Orchestrator task as `sa-ai-read`	Smoke-check results, Jira transition, graph re-sync, passport finaliser	Verified outcome, closed passport, Jira done, evaluation cases from any rework	None	A failed smoke check opens an incident to the owning pod, linked to the passport

Two details worth saying: the **Developer Agent never sees production**, and its identity cannot deploy anywhere but the DEV sandbox; and **step 06 and step 08 contain no agent at all**, which is where most of the business risk sits.

Each agent in detail

Every agent is a container image in Artifact Registry with four parts: a **runtime** (Codex or the Agents SDK), **instructions** (a versioned prompt), **skills** (pod-specific instructions and examples, e.g. Health Cloud or Marketing Cloud), and an **output schema**. It is activated only by the Dispatcher, runs once, and exits. Its tools come from the gateway and are limited to the step's `allowedTools`.

Agent	Runtime	Identity	Activated at	Reads	Produces
Orchestrator	Workflow code + Agents SDK classifier	`sa-orchestrator`	Run start, step 09	Jira story, graph	Classification (platform, pod, risk), close-out
Requirements	Agents SDK	`sa-ai-read`	Step 01	Jira, graph slice, Confluence via graph	`brief.json`
Architect	Agents SDK	`sa-ai-read`	Step 02	Graph tools, standards	`design.json`, Confluence draft

Agent	Runtime	Identity	Activated at	Reads	Produces
Developer	Codex	`sa-ai-dev`	Step 03, fix loop	Design, branch, graph	PR, `build-result.json`
Review	Agents SDK + static analysis	`sa-ai-read`	Step 04	PR diff, rules, graph	`findings.json`, PR comments
Test	Codex (code) + Agents SDK (cases)	`sa-ai-dev`	Steps 03, 04, 05	Design, code, graph coverage	Tests, Zephyr cases, `test-report.json`
Release	Agents SDK	`sa-ai-release`	Steps 05, 07	Copado, Git, graph	Promotions, `release-bundle.json`, CAB pack
Change Guardian	Rules engine (OPA + graph), no LLM in the verdict	`sa-guardian`	Step 02, every commit, every promotion, nightly	Diff, ownership graph, rulebook	PASS / NEEDS OWNER APPROVAL / BLOCK with reasons

Orchestrator Agent

Mostly not an LLM: it is the workflow definition plus a small classifier. At run start it reads the story and the graph slice and proposes platform (Health Cloud, Marketing Cloud, integration), owning pod and an initial risk tier, as structured JSON. Pod and platform pick the skills every later agent loads. At step 09 it finalises the passport and moves Jira to Done.

Requirements Agent

Input: the Jira story, linked issues, and `graph.context_slice`, which includes similar past stories and the Confluence rules that constrain the same objects. Instructions require: acceptance criteria in Given/When/Then, every statement citing a source id, and anything unsourced listed as an open question rather than assumed. Output is posted to the story as a comment and stored as `brief.json`. It cannot edit the story's fields.

Architect Agent

Runs the graph questions in a fixed order: `exists` (stop early if the thing exists), `find_reuse`, `blast_radius` on every object it would touch, `owners`, `classify_risk`, `coverage_gap`. Its skill holds Ascension's design rules (for example when Flow, Apex or LWC is the right tool, record-type entry criteria on shared objects, naming). Output:

`design.json` with components to create or modify, the reason for each, risk tier, test scope and affected pods, plus a Confluence draft for humans. Every dependency it lists must exist as a graph edge; the validator rejects any that do not.

Developer Agent: how it actually builds

1. **Workspace.** The container clones the repo, creates `mad/<story-key>` from the release branch, and authenticates the Salesforce CLI to the pod's **DEV sandbox** with the `sa-ai-dev` JWT. There is no credential for any other org in the container.
2. **Context.** Codex receives `design.json`, `changeplan.json`, the repo's instructions file (Ascension coding standards, naming, test rules) and the pod skill. It reaches the graph through MCP to look up existing classes, flows and fields instead of searching the repo blindly.
3. **Loop.** Codex edits metadata and code (Apex, triggers, Flow XML, LWC, permission sets, custom objects and fields), then runs, through allowed tools: `sf project deploy start --target-org dev` (or a check-only validation first), `sf apex run test` for the affected classes, and Salesforce Code Analyzer. It reads the errors and fixes, inside its budget of minutes and tokens.
4. **Commit.** Each commit runs the Guardian as a required check. When tests pass and coverage meets the threshold (Salesforce requires 75% for production; the pod may set higher), it opens the pull request and writes `build-result.json` with the commit SHA.
5. **Limits.** Shell commands are limited to an allowlist; the container has no route to production; Codex runs in its sandbox mode with network limited to the gateway and Git. If the budget runs out, it stops and the run pauses for the tech lead with what it tried.

Marketing Cloud variant: the same loop with the Marketing Cloud skill, deploying data extensions, SQL activities, journeys and content to the **Test business unit** through supported REST/SOAP APIs or the open-source SFMC DevTools (`mcdev`), and Data Cloud metadata through Data Kits where supported. Steps the APIs cannot do, such as campaign activation, are left as tasks for a person.

Review Agent

Runs deterministic checks first: Salesforce Code Analyzer (PMD rules such as SOQL or DML in loops, hardcoded ids), ESLint for LWC, permission-set checks (Modify All, View All). Then the LLM reviews the diff against the design and Ascension standards and

explains each finding with file, line and rule. It cannot approve; it writes `findings.json` and PR comments, and blocking findings start the fix loop.

Test Agent

In step 03 it writes test cases from the design, independently of the Developer Agent's code: Apex test classes and Jest tests (Codex), and Zephyr Scale test cases (Agents SDK, via the Zephyr API). In step 04 it runs them in DEV and picks regression scope from `blast_radius`, so a change to a shared object runs the tests of every pod that uses it. In step 05 it triggers Copado Robotic Testing and regression suites and collects results. Output: `test-report.json` bound to the SHA.

Release Agent

Step 05: creates or updates the Copado user story, commits the approved SHA's components, and promotes to SIT then UAT through Copado's API. Step 07: builds the manifest from the diff, maps environment-specific values, writes the runbook and a rollback plan (previous tagged version, destructive changes where needed), re-checks drift against the graph, and generates the CAB pack from the passport. It has no production credential; production is the owning team's pipeline.

Change Guardian

Not an LLM making the call: the verdict comes from **rules as code** (Open Policy Agent, Rego) evaluated over facts from the graph and the diff. An LLM may write the human-readable explanation, never the verdict. Checks: record-triggered flow on a shared object without record-type entry criteria; edits to shared Apex without tests for every pod that uses it; new field, validation rule or picklist value on a shared object without owner sign-off; hardcoded ids; SOQL or DML in loops; Modify All or View All; any change not from the pipeline. Architecture owns the rulebook in Git.

Controls, implemented

Four controls sit across every step. Each is its own small service so it can be reviewed and tested on its own.

Approval service

- **Where people approve:** an approval page (Cloud Run behind **Identity-Aware Proxy**, signed in with Ascension SSO) linked from Jira and a Teams or email notification. It shows the artifact, the evidence and the diff for that gate.
- **What is recorded:** approver identity, gate, run, scope, **commit SHA**, target environment, decision, comment, time, and an expiry. The record is signed with a **Cloud KMS** key and stored in Firestore and BigQuery.
- **How it resumes the run:** the service calls the Workflows callback endpoint for that run. Only this service's identity may call it.
- **Why approvals cannot drift:** every later step checks the approval it depends on. If the SHA, scope or target differs, the approval no longer matches and a new one is required.

Tool gateway and identities

All actions go through one **MCP gateway** (Cloud Run). For each call it checks four things with **Open Policy Agent**: the caller's service account, the tool, the target (org, environment, branch, project) and, for writes, a valid approval for the current SHA. Then it runs the action with that identity's own credential from **Secret Manager** and logs request, decision and result to BigQuery.

Identity	GCP service account	Can	Cannot
AI Read Service	`sa-ai-read`	Read the graph, Jira, Confluence; comment on stories; draft Confluence pages	Change code or any org
AI Development Service	`sa-ai-dev`	Branch and commit in Git; deploy and test in the pod's DEV sandbox	Touch SIT, UAT or production; merge to the release branch
AI Release Service	`sa-ai-release`	Copado user stories and promotions to SIT and UAT	Promote to pre-production or production
Guardian	`sa-guardian`	Read graph and diffs; set CI check status	Write code or deploy

Identity	GCP service account	Can	Cannot
Production	owning team's pipeline	Everything production	Not available to any agent

Each Salesforce identity is its own External Client App with a JWT certificate and a permission set scoped to its job, so least privilege is enforced in Salesforce as well as in GCP.

Failure and recovery

Failure	Detected by	What happens
Transient error (API timeout, 429, container lost)	`run_agent` retry policy	Retry with exponential backoff, up to 3 attempts; steps are idempotent (keyed by run, step, attempt)
Output fails its schema	Validator after each step	Agent re-runs once with the validation errors; then the run pauses for the step owner
Missing or stale context	Graph freshness check, agent open questions	Run waits in `NEEDS_INPUT`; the owning team is asked in Jira
Tests or review fail	Step 04	Fix loop to step 03, at most 2 rounds, then the tech lead decides
Budget breach (minutes, tokens, cost)	Dispatcher meter in Firestore	Task stopped, run paused, tech lead notified with the transcript
Approval expires or is rejected	Approval service	Run stops or returns to the step the approver names
Deployment fails	Copado (lower envs) or owner pipeline (production)	Lower env: back to 03; production: release owner runs the rollback plan from step 07

Failure	Detected by	What happens
Platform outage	Cloud Monitoring alerts	Workflows executions persist; the run resumes from its last checkpoint

Unprocessable Pub/Sub messages go to a dead-letter topic with an alert; nothing is dropped silently.

Accuracy guard

- Agents receive context only from `graph.context_slice`, never a raw org dump.
- Every output schema requires citations (node ids or source references); a validator checks that each cited node exists and that every listed dependency is a graph edge. Missing or invented → rejected.
- Existence answers come from a full lookup, never a sample. The POC learned this the hard way: a bounded search once cleared a field that already existed.
- The agent that builds never approves its own work: Review, Test and the Guardian are separate, and a person signs H3.

Security and compliance on GCP

The platform handles metadata, code and design text, never patient records; the controls below make that a property of the system rather than a promise.

Area	Control
Perimeter	One GCP project per environment (dev, pilot, prod-platform) in Ascension's landing zone; **VPC Service Controls** around Spanner, Cloud Storage, BigQuery, Firestore and Secret Manager
Network	Cloud Run services on private ingress; outbound only through **Cloud NAT** and **Secure Web Proxy** with a hostname allowlist: the OpenAI API, Atlassian, Salesforce My Domain hosts, Copado, the Git host
Encryption	**CMEK** with Cloud KMS on every data store; TLS in transit

Area	Control
Identity	One service account per role (read, dev, release, guardian, orchestrator, sync); no user keys; workload identity for Cloud Run; people sign in through Ascension SSO via Identity-Aware Proxy
Secrets	Salesforce JWT keys, Atlassian tokens, Copado and Git credentials in **Secret Manager**, readable only by the gateway and the sync jobs that need them
PHI	Metadata-only Salesforce access (no record read permission); **Sensitive Data Protection** de-identifies Jira, Confluence and Drive text before storage; a scheduled DLP scan of the graph; sandbox test data synthetic or sanitised
Model data	Only the bounded context slice is sent; secrets never enter a prompt; OpenAI terms (BAA, zero data retention) confirmed by Ascension security before pilot
Prompt injection	Source text is treated as data, not instructions; agents can act only through the gateway, which allows a fixed list of actions per identity and requires an approval bound to the SHA for any write
Supply chain	Images built by **Cloud Build**, scanned in **Artifact Registry**, admitted by **Binary Authorization**; dependencies pinned
Audit	Cloud Audit Logs, gateway decisions and every agent action in BigQuery, retained per Ascension policy
Production	No agent identity exists in any production org or pipeline

Change passport, observability and the learning loop

Change passport

One BigQuery table, `passport_entries`, one row per event: run, story, step, actor (agent identity or person), artifact URI, artifact SHA-256, commit SHA, approval id, graph `snapshot_id`, time, and the hash of the previous row for that run. Chaining each row to the one before makes the record tamper-evident: changing an old row breaks every hash after it. Artifacts themselves stay in Cloud Storage with object versioning. At step 07 the Release Agent renders the CAB pack from these rows, so CAB evidence is generated, not collated.

Observability

- **Cloud Logging** for every service and job, with `runId` on every line; **Cloud Trace** (OpenTelemetry) follows one run across services.
- **Cloud Monitoring** dashboards: sync freshness per source, runs by step and status, approval wait times, retries, budget spend, Guardian verdicts by rule.
- Alerts to the platform on-call for failed syncs, stuck runs, dead-letter messages and budget breaches.
- Delivery measures (cycle time per step, rework rounds, escaped defects) as BigQuery views, shown in Looker Studio.

Learning loop, without training a model on Ascension data

No model is fine-tuned. The system improves in four governed ways, each a change that is reviewed like code:

1. **Graph refresh.** Step 09 re-syncs the sources the change touched, so the next run sees the new component, test and release.
2. **Rulebook.** Each incident or near-miss is reviewed by architecture and, where it generalises, becomes a Guardian rule (Rego) with a test.
3. **Evaluation set.** Every rejected or reworked output is stored with the correct answer as an evaluation case. Prompt, skill or model changes must pass the full set in Cloud Build before they ship.
4. **Skills.** Pod skills (standards, examples, patterns) live in Git; improvements go through pull requests owned by the pod lead.

A model upgrade is therefore a configuration change that has to pass the same evaluations, which also answers lock-in: changing provider is the same path.

Tech stack

Everything except the model API and the delivery tools Ascension already owns is a managed Google Cloud service or an open-source library.

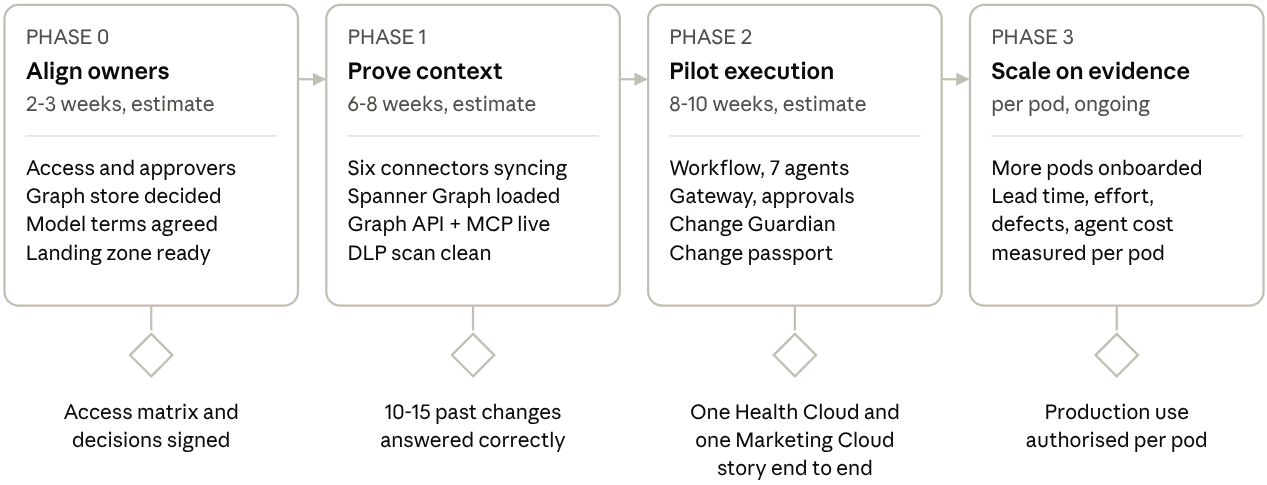
Layer	Technology	Used for
Scheduling and events	Cloud Scheduler, Eventarc, Pub/Sub	Sync cadences, source webhooks, step completion events, dead-letter topics
Batch compute	Cloud Run Jobs	Connectors, ingestors, graph loader, every agent task
Services	Cloud Run (FastAPI, Python 3.12)	Run API, Dispatcher, Graph API + MCP server, MCP gateway, Guardian, approval page
Workflow	Google Cloud Workflows	The nine-step run, retries, approval callbacks
Run state	Firestore (native mode)	Runs, checkpoints, budgets, sync high-water marks
Knowledge graph	Spanner Graph (Enterprise edition), ISO GQL	Nodes, edges, history, full-text and vector search
Snapshots and artifacts	Cloud Storage (versioned, CMEK)	Raw source artifacts, JSON snapshots, run artifacts
Audit and analytics	BigQuery, Looker Studio	Change passport, gateway decisions, delivery measures
PHI protection	Sensitive Data Protection (Cloud DLP)	De-identify free text before storage, scheduled scans
Security	IAM, VPC Service Controls, Cloud KMS, Secret Manager, Identity-Aware Proxy, Secure Web Proxy, Cloud NAT, Binary Authorization	Perimeter, keys, secrets, sign-in, egress allowlist, trusted images

Layer	Technology	Used for
Observability	Cloud Logging, Cloud Monitoring, Cloud Trace (OpenTelemetry)	Logs, dashboards, alerts, per-run traces
Platform delivery	Terraform, Cloud Build, Artifact Registry	Infrastructure as code, CI for the platform, scanned images
Coding agent	OpenAI Codex (Codex SDK / `codex exec`)	Developer Agent, test code
Reasoning agents	OpenAI Agents SDK (Python), OpenAI API	Requirements, Architect, Review, Release, classifier
Agent tools	Model Context Protocol (Python MCP SDK)	Graph tools and gateway tools exposed to agents
Policy	Open Policy Agent (Rego)	Gateway authorisation, Guardian rulebook
Validation	pydantic, JSON Schema	Every artifact and agent output
Salesforce tooling	Salesforce CLI (`sf`), Metadata API, Tooling API, Salesforce Code Analyzer, Jest, ESLint	Build, deploy to DEV, test, static analysis
Marketing Cloud tooling	SFMC REST / SOAP APIs, SFMC DevTools (`mcdev`), Data Kits	Build and deploy to the Test business unit
Delivery tools (existing)	Jira, Confluence, Git host, Copado, Copado Robotic Testing, Zephyr Scale	Stories, designs, code, promotion, test automation, test cases

Delivery plan, team, risks and open decisions

The build runs in four phases, each ending in a gate Ascension signs. Durations are planning estimates to confirm in Phase 0.

Four phases, each closed by a gate Ascension signs



delivery roadmap · 4 phases, 4 gates

Phase 2 starts only when Phase 1's gate shows the graph answers past changes correctly; durations are estimates to confirm in Phase 0.

Team (indicative)

Role	Count	Owns
Platform engineer (GCP)	2	Landing zone, Terraform, Cloud Run, Workflows, security controls
Data / graph engineer	1 to 2	Connectors, ingestors, Spanner Graph schema, Graph API
Agent engineer	2	Agent containers, prompts and skills, evaluation set, gateway tools
Salesforce tech lead	1	Pod skills, Salesforce identities, Copado integration, Guardian rules with architecture
QA automation engineer	1	Test Agent skills, Zephyr and Copado Robotic Testing integration
Ascension partners	part time	Architecture, security, pod leads, release owner

Risks and mitigations

Risk	Mitigation
Graph misses a dependency (for example Flow internals)	Tooling API flow reader in Phase 1; coverage report per pod; an unproven link never clears a change
Model output quality varies	Schema validation, citations, independent review and tests, human gates, evaluation set before any prompt or model change
Security approval for the model endpoint takes long	Start the OpenAI terms review in Phase 0; keep a Vertex AI model as a fallback behind the same interface
Copado or Marketing Cloud APIs cannot do a step	Leave that step to a person and record it in the passport; do not script around the platform
Approvers become the bottleneck	Approval page shows exactly the evidence for that gate; low-risk changes route to one approver by risk tier
Cost overrun on model usage	Per-run budgets in the Dispatcher; graph answers most questions without a model call; spend dashboard

Open decisions for Phase 0

- [ ] Graph store: confirm Spanner Graph Enterprise, or Semantica if already licensed (behind the same Graph API)
- [ ] Model access: OpenAI terms (BAA, zero data retention) or a Vertex AI model
- [ ] Git host and CI system, which decide how the Guardian becomes a required check
- [ ] Copado edition and whether pre-promotion quality gates can call an external API
- [ ] Which pilot stories: one Health Cloud (Network Services) and one Marketing Cloud (Nudge)
- [ ] Approver list per gate and risk tier
- [ ] Tooling API client-credentials setup in the sandbox (the POC's open blocker)

Presenter cheat sheet

One-line answers, each pointing at the section that backs it.

If they ask	Say
What does it run on?	Google Cloud, inside Ascension's landing zone: Cloud Run, Cloud Workflows, Spanner Graph, Firestore, BigQuery. All managed services.
Which graph, and why not Semantica?	Spanner Graph: durable, shared, audited, inside the perimeter. Semantica is an in-memory library; we keep its API shape as our contract.
Who runs the schedules?	Cloud Scheduler starts each connector as a Cloud Run Job; source webhooks start them early through Pub/Sub.
How does an agent know it's its turn?	It doesn't. The workflow launches one agent task per step with typed inputs and waits for a typed output.
How does the Developer Agent build?	Codex in an isolated container, on its own branch, deploying only to the DEV sandbox, looping on deploy, tests and static analysis until green, then a pull request.
Can it reach production?	No. No agent identity exists in any production org or pipeline; the owning team deploys after CAB.
What if it's wrong?	Cited context only, schema checks, independent review and tests, a fix loop, then people at H3.
What if a step breaks?	Retry with backoff, then pause and escalate; the run resumes from its last checkpoint.
How does it learn?	Graph refresh, Guardian rules, an evaluation set and pod skills. No model trained on our data.
PHI?	Metadata only from Salesforce; DLP de-identifies text before storage; only a bounded slice goes to the model.

If they ask	Say
Lock-in?	Tools behind MCP, a vendor-neutral JSON snapshot, prompts versioned: a model switch is config plus evaluations.
What's proven today?	The sandbox graph POC (2,177 nodes, 2,786 edges; duplicate-field detection) and six of seven agents built from agents in use. The GCP runtime is the build.

Sources

Product facts checked by web search on 11 Oct 2026 (the official pages could not be opened from this environment, so confirm in Phase 0): Spanner editions announcement (Spanner Graph, full-text and vector search in Enterprise edition), Spanner Graph (ISO GQL), Spanner Graph schemaless data, Workflows callbacks (default 12-hour wait, timeout configurable), OpenAI Codex SDK, via Braintrust's integration docs (threads, structured output, sandbox modes). POC figures: `poc/HANDOFF.md` on branch `claude/funny-babbage-t29499`.